

BIG 5

Naledi Methula - 37828118 (GROUP LEADER)

Bongiwe Mncube- 36149829

Sithembiso Masoka- 39069656

Seisa Satekga - 32395620

Thando Kunyuza- 48124095

SUBMISSION DATE: 31/08/2025

Table of Contents

Contents

1.	System Overview Purpose & Vision	3
2.	Problem Definition & Business Objectives Key Challenges	3
	Proposed Solutions	3
3.	Conceptual Design Highlights	3
	High-Level Architecture	3
4.	Constraints & Design Rationale Constraints	3
5.	Expected Outcomes:	3
	Summary of source Files	8
2.	Hierarchy analysis	9
1.	Coverage	12
2.	Correctness	12
3.	Performance	13
4.	Scalability	13
5.	Governance & Usability	13
	CONCEPTUAL NOSQL DESIGN	14
	Collections include: sales, customers, deposits, and products.	14
	NoSQL Specifics	14
	Sales Document:	15
	Product Document:	16
	Data Sources	18
	ETL Process	18
	NoSQL Data Warehouse (MongoDB)	18
	BI / Analytics Dashboard	18
	NoSQL Consumption Layer	18

1.EXECUTIVE SUMMARY

1. System Overview

Purpose & Vision

ClearVue Ltd. is seeking to modernize its overall sales performance analytics by implementing a NoSQL-based Business Intelligence (BI) system that is capable of supporting its complex reporting (daily, weekly, monthly, quarterly, and annually)and also aligned to each unique financial year.ClearVue Ltd. currently relies on MicroSoft Access which limits agility, scalability, and timely insights which prevents the strategic and supplier-focused analytics

The BIG 5 proposed solution of the NOSQL- based system will address limitations of the current Microsoft Access based system by implementing real-time data integration, hierarchical and semi-structured data support, and flexible reporting aligned to the ClearVue Ltd. unique financial year.

2. Problem Definition & Business Objectives

Key Challenges

- Inadequate temporal alignment: Robust querying and reporting of sales must update to a financial month beginning the last Saturday and ending the last Friday, which current systems cannot accommodate.
- Limited flexibility & scalability: Microsoft Access cannot effectively handle diversified and evolving data structures, namely supplier hierarchies and product categories.
- Inefficiency of real-time insights: Hindered reporting prevents timely decision-making and supplier-focused analytics readiness.

Proposed Solutions

- Enable multi-level granularity of sales data querying and reporting basis (daily, weekly, monthly, quarterly and annually) that update on financial months beginning the last Saturday and ending the last Friday, which aligns with the custom fiscal calendar and supports sales performance analysis.
- Integrate NoSQL-based database to support hierarchical and semi-structured data.
- Support flexible, hierarchical and semi-structured data models for updating and adapting to evolving analytics needs.
- Incorporate streaming data to support instantaneous insights on transaction data of payments using Apache Kafka.
- Offer user-friendly BI dashboards for visualization (via Power BI, Tableau,custom UI).
- Upgrade MS Access which will result in limited improvement, However structural rigidity and scaling issues will remain.

3. Conceptual Design Highlights

- **Key Capabilities:**

Time-based reporting basis (Daily,Weekly,Monthly,Quartely and Annually),enable Supplier analytics readiness,provide support for hierarchical product and new data types and lastly perform streaming integration.

High-Level Architecture

- A flexible NoSQL platform such as Mongo DB serves as the core repository for the structured, semi-structured and hierarchical data.
- A streaming layer such as Apache Kafka will process real-time payment and transaction data.
- An analytical and reporting layer to integrate the data from the NoSQL store and streaming system to generate aligned and timely reports.
- A BI visualization layer will present data through dashboards while reporting for key stakeholders.)

4. Constraints & Design Rationale

Constraints

- Custom fiscal calendars demand precise time periods.
- Data structures evolve with focus on supplier analytics which require adaptable schemas.
- Streaming integration must have minimal disturbance on the overall system performance.

5. Expected Outcomes:

Overall faster queries on critical reports, improved decision-making, scalable model for new data types, operational dashboards for real-time visibility.

- **Temporal flexibility:** Tailored reporting aligned with ClearVue’s financial month (last Saturday to last Friday) ensures precise temporal alignment across all report types.
- **Time Alignment:** Accurately aligned reports and faster queries on critical reports across all time windows will enhance strategic decision-making and forecasting.
- **Flexible Data Modeling:** Future-proofs the system against evolving supplier and product analytics needs.
- **Scalability & Flexibility:** NoSQL architecture and modular layers offer growth potential and agility enabling ClearVue long-term innovation.
- **Temporal flexibility:** Tailored reporting aligned with ClearVue’s financial month (last Saturday to last Friday) ensures precise temporal alignment across all report types.
- **Flexible data modeling:** Document stores or similar NoSQL approaches enable agile handling of hierarchical or evolving data without rigid schemas.
- **Real-time analytics:** The integration of streaming tools will enable instantaneous incorporation of transactional data into analytical workflows.
- **Modern operational dashboards:** Use of BI visualization platforms will enhance reporting clarity and user engagement.
- **Advantages over relational systems:** Compared to traditional systems like Microsoft Access, NoSQL will offer superior scalability, agility while supporting diverse and evolving data types.

2.Requirements Definition Approach

Given the project constraints, requirements will be obtained through an analysis of the provided data files and the application of industry knowledge, as direct access to internal stakeholders is not available. As external proposers, our approach compensates for the lack of direct stakeholder access with data archaeology and validation against industry standards.

Stakeholders Involved

The following stakeholder groups are considered in this analysis:

- Executive Management: Requirements defined through the strategic goals in the RFP.
- Sales & Finance Departments: Needs derived from data analysis and functional role-play.
- IT/Data Engineering Team: Requirements gathered from the RFP and validated through technical benchmarking.
- Industry Best Practices: Acting as a proxy for external consultants, guiding technology and design choices.

- Primary method: Artifact analysis & Data archaeology

The 17 provided data files (e.g. Customer, Payment Headers/Lines, Sales Headers/Lines, Product categories, etc.).

- Data structures: Identifying entities, attributes, primary keys and foreign keys to map the existing operational landscape.
- Business rules: Inferring rules from relationships. For example, the link between **Sales line** and **products** defines a core business process. This will be achieved through analysis of the foreign keys with all 17 data files.
- Hierarchy Identification: A specific focus will be on identifying hierarchical structures within the data (e.g. Product categories → Product ranges → Products → Product styles) to inform the flexible schema design of the NoSQL database.

Team task: The provided files will be divided for profiling. Each member will take 3-4 files to document by their structure, sample data, and hypothesize their business purpose. Findings will be consolidated to form a unified source system analysis.

- Secondary method: Analysis of the RFP & Strategic Directive

The RFP document is not just a brief but a definitive statement of strategic intent from “Executive Management”. We will analyse it to derive high level objectives:

- Explicit Needs: The mandatory requirement for multi-timeframe reporting (daily, weekly, monthly, quarterly, annual) aligned with the unique financial calendar.
- Implicit Needs: The directive for “supplier analytics readiness” implies future requirements for analysing Suppliers, Purchases Headers/Lines, and their performance metrics.
- Non-Functional Requirements: The emphasis on a “flexible data model”, “scalability”, and “real-time insights” defines the technical architecture constraints.

- Validation through Research and Prototyping

To bridge the gap left by unavailable stakeholders, we will validate our inferred requirements against external sources.

- Industry best Practices as a stakeholder: research into NoSQL BI patterns will validate our data model choices. For instance, the need for rapid aggregation across custom time periods will justify using a pre-aggregation or bucket pattern.
- Functional Role Play: We will assign team members to advocate for the needs of implicit stakeholder roles:
 - A “Sales Manager” would require reports on sales performance by **Customer Region** and **Representative**. This need will be validated by ensuring the **sales_header**, **sales_line**, and **customer_regions** data can be joined and aggregated efficiently.
 - A “Finance Officer” would require the **Age Analysis** report and accurate period-closing based on the financial calendar. This need will be validated by mapping the **age_analysis** file and linking it to the **customer** and **payment** data.
 - A “Procurement Manager” would require analytics on supplier performance and purchase trends.
 - IT/Data engineers: Needs are defined by the RFP's emphasis on scalability, flexibility, and maintainability, validated through technical research.
 - Supplier Relations Managers: Future needs are proactively defined by analyzing the purchases and suppliers data files.
- Real-time data integration architectures: Tools like Apache Kafka will be vital to research into data integration architectures..
- Design Spikes: We will build rapid prototypes for critical, high-risk areas specifically, testing different methods for calculating the financial calendar to validate the feasibility of our proposed solutions.

- Benchmarking & Technical Spike Prototyping:

Industry best practices act as our consultant stakeholder.

- NoSQL Pattern Research: We will test patterns like the Bucket Pattern (for time-series payment data) and Extended Reference Pattern (for denormalizing product brands in sales documents) against our data model.
- Financial Calendar Logic Proof-of-Concept: A technical spike will be conducted to build the algorithm that converts **TRANS_DATE** into the **FIN_PERIOD**, validating this core requirement is achievable.

- Iterative Validation via Conceptual Modeling: Requirements will not be static. They will be validated by building and critiquing high-level conceptual models of the NoSQL collections. For example, a draft document model for a **sales** collection will be presented to the "Sales Manager " to confirm it contains all necessary data for their reports.

This approach is necessary because it transforms a limitation, lack of direct stakeholder access, into a strength. By treating the data itself and the RFP as primary sources of information, we ensure our requirements are grounded in reality and directly aligned with ClearVue's stated strategic goals. The use of role-play and prototyping mitigates the risk of building based on incorrect assumptions, ensuring the final design is both technically sound and business-relevant.

Research acknowledgements:

Resources from official documentation sites such as MongoDB.com, Apache Kafka Documentation, and technical whitepapers from leading cloud providers (AWS, Azure) will be consulted. Insights from validated community resources like Stack Overflow will be critically evaluated and used to inform solution design.

AI-Assisted Analysis:

AI tools will be used as a force multiplier for initial research, such as generating explanations of complex data modelling patterns or drafting preliminary summaries. Crucially, every AI-generated output will be critically evaluated and stress-tested by the team. For example, an AI-suggested schema will be challenged with complex query scenarios to ensure it meets the reporting needs we've derived. All usage will be meticulously documented in the AI Usage Log.

3.Business Requirements

Requirements	Detailed Descriptions
Comprehensive Sales Data Reporting	The system must produce accurate and automated sales reports across all timeframes (daily, weekly, monthly, quarterly, annual). Reports must allow drill-downs (e.g., region, product, customer segment) and comparisons over time, enabling management to evaluate performance from both operational and strategic perspectives.
Alignment with Financial Calendar	ClearVue's financial year follows a unique cycle (last Saturday to last Friday). The system must strictly adhere to this definition to avoid misrepresentation of performance trends, ensuring that results align with internal accounting and strategic review cycles.
Future-Ready Supplier Analytics	The solution must be extensible to incorporate supplier performance data and supplier-related KPIs. This includes supporting new datasets, hierarchical relationships (e.g., suppliers by region or category), and integration with procurement data, ensuring a smooth transition when supplier analytics become a core focus.
Adoption of NoSQL Database	A NoSQL solution (MongoDB) must be implemented to manage semi-structured and hierarchical data efficiently. The solution must support dynamic schemas, large volumes of unstructured data, and faster query performance compared to traditional relational systems like MS Access.
Streaming Data Integration	The platform must integrate with streaming technologies (e.g., Apache Kafka, AWS Kinesis) to support real-time ingestion of data such as payment transactions. This will allow the BI system to reflect up-to-the-minute insights, improving responsiveness and operational decision-making.
Interactive BI Dashboard & Visualization	A user-friendly BI dashboard (Power BI, Tableau, or custom solution) must be implemented, offering intuitive visualizations, drill-down functionality, and customizable KPI tracking. The dashboard should empower managers to explore insights independently without relying heavily on IT staff.
Scalability and Flexibility	The system must accommodate data growth (both in volume and complexity) without major redesigns. Scalability should extend across storage, processing, and reporting layers, ensuring ClearVue can support future requirements such as predictive analytics and machine learning.
Data Quality, Integrity, and Security	The BI system must enforce strict data quality standards, ensuring accuracy, completeness, and consistency across reports. In addition, the system must integrate access control and security protocols, ensuring sensitive data is only visible to authorized personnel.
Ease of Use and Adoption	Interfaces must be intuitive for non-technical managers, minimizing the need for specialized training. Adoption success will depend on providing simple navigation, clear visualizations, and consistent user experience across devices (desktop and mobile).

4.High-Level Business Objectives

Objectives	Detailed Descriptions
Enable Evidence-Based Decision Making	Provide management with fast, accurate, and reliable insights that support both tactical and strategic decisions. The system should reduce reliance on manual spreadsheets and anecdotal evidence by ensuring that decisions are data-driven and verifiable.
Increase Organizational Agility	Establish a BI environment that is adaptive to evolving business priorities, including changes in product categories, customer behaviors, and future focus on supplier analytics. Agility ensures that ClearVue can pivot quickly without costly system overhauls.
Enhance Sales Performance Analysis	Deliver precise sales performance reporting aligned with ClearVue’s unique financial year. This ensures that management can identify true business cycles, monitor seasonal trends, and measure performance in a way that directly aligns with strategic planning and forecasting.
Prepare for Supplier-Centric Analytics	Build a BI system capable of transitioning seamlessly into supplier performance tracking. This will enable ClearVue to strengthen supplier relationships, negotiate better contracts, and optimize procurement strategies.
Introduce Real-Time Intelligence	Integrate live streaming data to provide immediate visibility into sales and financial transactions. Real-time analytics will give managers the ability to identify issues (e.g., sudden sales drops, payment delays) and respond proactively.
Secure a Competitive Advantage	Leverage modern BI tools and NoSQL technologies to outperform competitors still reliant on rigid relational systems. ClearVue will benefit from faster reporting, greater flexibility, and deeper insights, positioning it ahead in the marketplace.
Boost Operational Efficiency	Automate reporting and reduce manual intervention in data preparation. This will save time, minimize errors, and enable staff to focus on analysis rather than repetitive data handling.
Support Long-Term Strategic Growth	Lay the foundation for future enhancements, including predictive analytics, AI-driven insights, and advanced visualization. The system must not only solve today’s reporting challenges but also act as a scalable platform for tomorrow’s innovation.

5.ANALYTICAL AND INFORMATION REQUIREMENTS

Report Name	Purpose	How	Key Metrics	Data Sources (Dimensions)	Reasoning
Daily Sales	Track daily transactions by representative, channel, and transaction type	Aggregate transactions by date and customer category; include transaction type; sum	Sales Value, Number of Transactions, Commission, Split by REP & TRANSTYPE	Transactions Table, Customer Master, Transaction Types Table	Monitor daily sales patterns, identify high-performing customer categories, channel monitoring, reconciliation
Regional Sales	Analyze sales by rep region, customer segment, and geography	Aggregate sales and AR balances by region/ rep; calculate trends, overdue %, top	Sales Value per Region (BJ, BM, RL), House vs Wholesale vs Dept Store split, Contribution %	Sales Transactions, REP Master (region tags)	Support regional sales strategy, resource allocation, evaluate operational efficiency, guide management
Age Analysis Report	Monitor outstanding customer balances by age buckets (30/60/90+ days)	Use AR aging data; calculate % overdue; rank customers by overdue amounts;	Balances in 30/60/90/120+ buckets, Split by Customer Type, Flagged problem accounts (ZZZ codes)	Accounts Receivable, REP Master, Customer Type	Support credit control, cash flow management, bad debt monitoring
Supplier Spend & Concentration	Track purchases per supplier with associated spend	Aggregate purchase amounts per supplier; sum monthly and total spend; highlight	Total Purchase Value, No. of Orders, Supplier Category, % of Total Spend by Supplier, Top 5/10 suppliers	Supplier Master, Purchase Transactions Table	Monitor procurement spend, evaluate supplier performance, manage risk, reduce supplier dependency,
Monthly Product Trends	Identify Top 10 products and trends over time by revenue per category and range	Aggregate revenue per product & category per month; rank top 10 by revenue; include	Revenue by Product/ Category, Units Sold, MoM Growth %, Top 10 Products (Pareto 80/20)	Transactions Table, Product & Time Dimensions	Identify high-revenue products; plan inventory and promotions
Customer Analysis	Track customer sales, identify outstanding balances, and trend analysis	Merge AR Aging + Transactions + Customer Master; calculate % overdue;	Cumulative Sales, % Overdue, Outstanding Balances by Customer	Customer Master, AR Aging Table, Transactions Table	Monitor customer performance, credit exposure, and overdue balances
Debtors & Bad Debt	Track overdue balances & problematic accounts	Identify flagged accounts, calculate overdue %	Outstanding Balances, Overdue %, Flagged Bad Credit (ZZZ4, ZZZP), Closed Accounts	Accounts Receivable, Customer, REP Master	Protect cash flow, reduce financial risk
Deposit Reconciliation	Ensure cash collected is applied correctly; reconcile payments	Join deposits with transactions per customer; sum deposits per period; compare	Deposits Collected, Discrepancies, Reconciled Balances	Deposits Table, Transactions Table, Customer Master	Ensure accurate application of payments, detect discrepancies
Exception & Risk Report	Detect problem accounts, unknown codes, and risky transactions	Identify transactions with unknown or problematic codes, discounts only	Transactions with 99999 Unknown, Do Not Use/ Problem/Bad Debt (ZZZ), Discount-only Transactions	Sales Transactions, REP Master, TRANSTYPE Master	Data quality assurance, revenue assurance, credit risk monitoring

6.PRELIMINARY SOURCE SYSTEM ANALYSIS

This analysis provides a high level overview of the 17 provided data files, identifying key entities, their purposes, primary keys, potential relationships and notable data characteristics. This profiling forms the foundation for designing the NoSQL data model.

Summary of source Files

The provided files can be grouped into several core business domains:

1. Sales & Payments: Sales_Header, Sales_line, Payment_headers, Payment_lines
2. Products: Products, Product_brands, Product_catagories, product_ranges, product_styles
3. Customers: Customer, Customer_categories, Customer_regions, Customer_account_parameters
4. Procurement: Purchase_Headers, Purchase_lines, Suppliers
5. Supporting: Age_analysis, Representatives, Trans_types.

Parent Entity	Child Entity	Relationship Type	Cardinality	Linking Key(s)	Notes
Customer	Sales Header	One-to-Many	1 : M	CUSTOMER_NUMBER	A single customer can have many invoices.
Sales Header	Sales Line	One-to-Many	1 : M	DOC_NUMBER	A single invoice can have many line items. Prime candidate for embedding in
Product	Sales Line	One-to-Many	1 : M	INVENTORY_CODE	A single product can appear on many invoice lines.
Customer	Payment Headers	One-to-Many	1 : M	CUSTOMER_NUMBER	A single customer can make many payments.
Payment Headers	Payment Lines	One-to-Many	1 : M	DEPOSIT_REF	A single payment deposit corresponds to one summary line. Likely to be
Supplier	Purchases Header	One-to-Many	1 : M	SUPPLIER_CODE	A single supplier can have many purchase orders.
Purchases Header	Purchases Lines	One-to-Many	1 : M	PURCH_DOC_NO	A single purchase order can have many line items.
Product	Purchases Lines	One-to-Many	1 : M	INVENTORY_CODE	A single product can be ordered on many purchase orders.
Representative	Sales Header	One-to-Many	1 : M	REP_CODE	A single sales rep can be associated with many invoices.
Representative	Customer	One-to-Many	1 : M	REP_CODE	A single sales rep can manage many customers.
Trans Type	Sales Header	One-to-Many	1 : M	TRANSTYPE_CODE	A transaction type (e.g., Invoice) can be used on many sales headers.

Critical Relationship Mapping

Analysis of primary and foreign keys revealed a highly normalized operational database schema. The following key relationships were identified

- Sales Transaction Lifecycle: A **sales_header** (invoice) has a one-to-many relationship with **sales_line** items. These invoices are paid by **payment_lines** applications, which are grouped under a **payment_headers** record.
- Product Hierarchy: The **products** table is a central hub linked to multiple descriptive tables (**product_brands**, **product_categories**, **product_ranges**, **product_styles**), creating a complex snowflake schema.

Customer Structure: The **customer** table is linked to geographic (**customer_regions**) and segmentation (**customer_categories**) tables, and has extended attributes in **customer_account_parameters**.

2. Hierarchy analysis

Hierarchy Root	Hierarchy Description	Levels (Inferred from Keys)	Implication for NoSQL
Product	Product Classification	Product -> Product Category -> (Brand + Range)	Strong case for denormalization. A product document should embed all its hierarchical data (category, brand, range descriptions) for optimal read performance.
Customer	Geographical & Segmentation	Customer -> Customer Region & Customer Category	Similarly, a customer document should embed its REGION_DESC and CCAT_DESC to avoid joins during analysis.
Sales	Transaction Composition	Sales Header -> Sales Line Items	The classic parent-child relationship. Must be embedded in NoSQL to form a complete sales transaction document.

Two primary hierarchies were discovered, which are crucial for analytical reporting and NoSQL design:

- Product Hierarchy: **Product Brand -> Product Category -> Product Range -> Product -> Products Style.**
- Customer Hierarchy: **Sales Region -> Customer Category -> Customer -> Customer Account Parameters.**

3.Cardinality assessment

Entity	Estimated Cardinality	Reason
sales_line	Very High	Transactional; every sale creates one or more records.
payment_lines	High	Transactional; every payment creates a record.
products	Medium	Master data; number of unique items sold.
customers	Medium	Master data; number of active clients.

4. Key Findings & Implications for the BI System

- Existence of a Financial Period Key: The **FIN_PERIOD** field found in sales_header, **payment_lines**, and **age_analysis** is a critical discovery. It indicates the complex financial calendar logic is pre-calculated in the source system, significantly simplifying ETL development and ensuring temporal consistency across all reports.
- Presence of Semi-Structured Data: Files like **product_styles** (with attributes like **GENDER**, **MATERIAL**) and **customer_account_parameters** (key-value pairs) contain variable data not suited for rigid SQL schemas. This strongly justifies the NoSQL approach for its ability to handle flexible, embedded documents.
- Data Quality Notes:
 - Decimal Formatting: Numeric fields in payment_lines (e.g., BANK_AMT) use commas as decimal separators (-2046,3), requiring transformation.
 - Code Mismatch: A potential inconsistency exists between product_categories.BRAND_CODE (values: 6,7,9) and product_brands.PRODBRA_CODE (values: 1,2,3,4), requiring investigation during ETL.
- Report vs. Source Data: The age_analysis file is a pre-calculated report output (containing bucketed amounts like AMT_30_DAYS), not a source transaction table. The new BI system must replicate this calculation logic from the underlying sales and payment data.

5. Conceptual NoSQL collection outline

Based on this profiling, the BI database would likely be built from these main collections:

- 1.Sales: Each document is a complete invoice (sales_header data) with an embedded array of its sales_line items. Frequently accessed dimensions (e.g. product_name, customer_name) are denormalised into the document.
- 2.Payments: Documents represent a payment (payment_lines data), potentially merging the sparse header information.
- 3.Products: Each document is a product (products data) with embedded sub-documents for its category, brand, range and attributes (from product_styles).
- 4.Customers: Each document is a customer (customer data) with embedded region_description, category_description and an array of their account_parameters
- 5.Financial_calendar: A separate collection generated from the Fin_period logic, storing start/end dates for each period, enabling easy temporal querying.

This structured profiling provides a clear, justified roadmap for the subsequent conceptual design.

7.PRELIMINARY SUCCES CRITERIA

1. Coverage

This refers to how well the prototype addresses the range of reporting and dashboard needs identified in the requirements. It ensures that all critical business scenarios (e.g., daily sales, fiscal month reports, product and supplier performance, payments overview) are implemented in the prototype.

Implement ≥ 4 priority dashboards: Daily Sales, Financial Month Performance, Product & Supplier Trends, Payments Overview (streaming).Each dashboard must include: global filters (Date/Fiscal Period, Region/Store, Supplier, Product)

Implement ≥ 4 priority dashboards:Daily Sales (real-time)Financial Month Performance (Sat→Fri alignment)

Product & Supplier Trends (drill-down enabled)Payments Overview (streaming transactions)

Query Performance

This specifically focuses on the efficiency of database queries behind the dashboards. Even if visuals are attractive, if queries run slowly or time out, the BI system fails. Query performance ensures the NoSQL design (indexes, aggregations, sharding) supports fast data retrieval even under load.

Measures efficiency of ad-hoc queries on sales and payments data.

Target: Ad-hoc queries return results ≤ 5 seconds for datasets ≤ 1 million rows. Complex queries (e.g., cross-period product performance) ≤ 15 seconds.Measurement: Query execution time logs.

User Satisfaction

This measures how well the BI prototype meets the expectations, preferences, and confidence levels of end-users (e.g., managers, finance staff, analysts, supplier liaisons). Even if dashboards are correct and fast, if users find them confusing, unattractive, or unhelpful, adoption will fail. User satisfaction ensures the BI system delivers value in the day-to-day workflow of its intended audience.

Measures overall value and acceptance of the prototype by its intended users.Target: ≥ 80% of surveyed users rate satisfaction ≥ 4/5 on usefulness, speed, and reliability.Target: ≥ 75% of managers confirm BI reports improve decision-making.

Measurement: Post-pilot surveys, stakeholder interviews, feedback forms.

2. Correctness

Correctness ensures that the BI system produces accurate and reliable results, especially with ClearVue’s unique fiscal calendar. It covers whether data transformations, fiscal date mapping, and reconciliation with existing financial figures are correct and consistent.

Fiscal calendar validation: 100% of test periods map correctly (e.g., February 2025 = Jan 31 → Feb 27).

Finance reconciliation: Variance between prototype and legacy totals ≤ 0.5% on sampled months.

3. Performance

Performance refers to how quickly and efficiently the prototype responds to user interactions and processes data. This includes dashboard responsiveness, query execution time, aggregation refresh speed, and streaming latency.

Dashboard responsiveness: Top 5 dashboards render visuals in ≤ 3 seconds when filtering by Store/Region/Product for the last 3 months.

Batch loads: summaries refresh in ≤ 10 minutes post-load.

Streaming pipeline: new payments visible in BI dashboards ≤ 5 minutes end-to-end.

4. Scalability

Scalability measures whether the prototype can handle increased data volumes and more users without breaking or slowing down excessively. ClearVue expects data growth (millions of records) and more users (finance, supplier, regional managers), so the system must show it can expand beyond the small prototype.

Concurrent users: Prototype sustains ≥ 10 BI users running queries simultaneously without timeout.

Data growth test: Handles synthetic expansion to ≥ 5 million sales records without query failures.

Target: Handles ≥ 5 million sales records without query failures.

Measurement: Load testing with synthetic data.

5. Governance & Usability

Governance ensures the system manages data integrity, KPI consistency, access control, and audit logging. Security ensures only the right people see the right data. Without governance, managers may use inconsistent definitions (e.g., “net sales”), leading to conflicting reports.

KPI Dictionary: Approved by stakeholders, covering ≥ 10 key sales and supplier KPIs.

Access Controls: Role-based access implemented (e.g., Finance vs. Sales vs. Supplier teams).

Audit Logging: Enabled for all BI queries and dashboard accesses.

Reliability & Data Quality Reliability ensures the prototype BI system is stable, available, and trustworthy. Data quality ensures inputs are valid (no duplicate transactions, no negative sales amounts unless refunds). Users must trust the dashboards to reflect reality, or adoption will fail.

- **Target:** $\geq 99\%$ data load success rate without errors.

- **Target:** Data validation rules applied to 100% of imports (e.g., no nulls in primary keys, consistent product codes).

- **Measurement:** ETL logs, automated data validation checks.

8.Conceptual NoSQL Design

CONCEPTUAL NOSQL DESIGN

The schema is intended to support ClearVue Ltd.'s distinct financial calendar, hierarchical product categories, evolving supplier analytics, and near-real-time payment event ingestion. We propose a document-oriented schema using MongoDB. The Hierarchical data such as line items and customer deposits will be embedded to avoid excessive joins. This model reduces redundancy and query complexity, while supporting ClearVue’s analytics and BI dashboards with real-time aggregation. This model avoids data redundancy by segregating slowly changing master data (products, brands, and suppliers) and inserting high-frequency line items in transactions to improve read performance. It meets ClearVue's reporting needs for daily, weekly, monthly, quarterly, and annual views, following the financial year rule (last Saturday → last Friday).

Collections include: sales, customers, deposits, and products.

- Sales_transactions (header and embedded line items).
 - Product (SKU master with hierarchical categories and adjustable attributes)
 - Reference documents for brand and supplier information.
- Customers_core and customers_contact (separated for PII governance).
- Payments_events (append-only time-series of tenders).
- Stores and regions (geo-hierarchy for rollups).
- fiscal_calendar (materialized keys and boundaries for joins and aggregations)

NoSQL Specifics

- Indexes:
 - sales_transactions: {txnDateTime}, {financialKeys.monthKey}, {storeID}, {lineItems.productID}
 - products: {productID}, {brandID}, {supplierID}, {categoryPath}
 - payments_events: {eventTs}, {tenderType}, {financialKeys.monthKey}
- Sharding strategy: By {financialMonthKey, storeID} for balanced distribution.
- Schema flexibility: Product attributes stored as attributes{} for evolving fields.
- Security: PII separated into customers_contact with field-level encryption.

Sales Document:

```
{
  "salesID": "S12345",
  "txnDateTime": "2025-07-01T12:30:00Z",
  "storeID": "ST001",
  "channel": "Online",
  "customerID": "C987",
  "transType": "TAX INVOICE",
  "financialKeys": {
    "dayKey": "2025-07-01",
    "weekKey": "2025-W27F",
    "monthKey": "2025-M07",
    "quarterKey": "2025-Q3",
    "yearKey": "FY2025"
  },
  "lineItems": [
    {
      "productID": "P1226COL",
      "quantity": 10,
      "unitPrice": 50,
      "categoryPath": ["Fitness", "Bands", "Resistance"],
      "discount": 20,
      "extended": 480
    }
  ],
  "totals": {
    "gross": 500,
    "discountTotal": 20,
    "net": 480
  }
}
```

Product Document:

```
{
  "productID": "P1226COL",
  "name": "TigerBand Pro",
  "brandID": "B101",
  "supplierID": "SUP07",
  "categoryPath": ["Fitness", "Bands", "Resistance"],
  "attributes": {
    "colour": "Black",
    "size": "Medium",
    "bluetooth": true
  },
  "status": {"active": true, "startDate": "2025-01-01"}
}
```

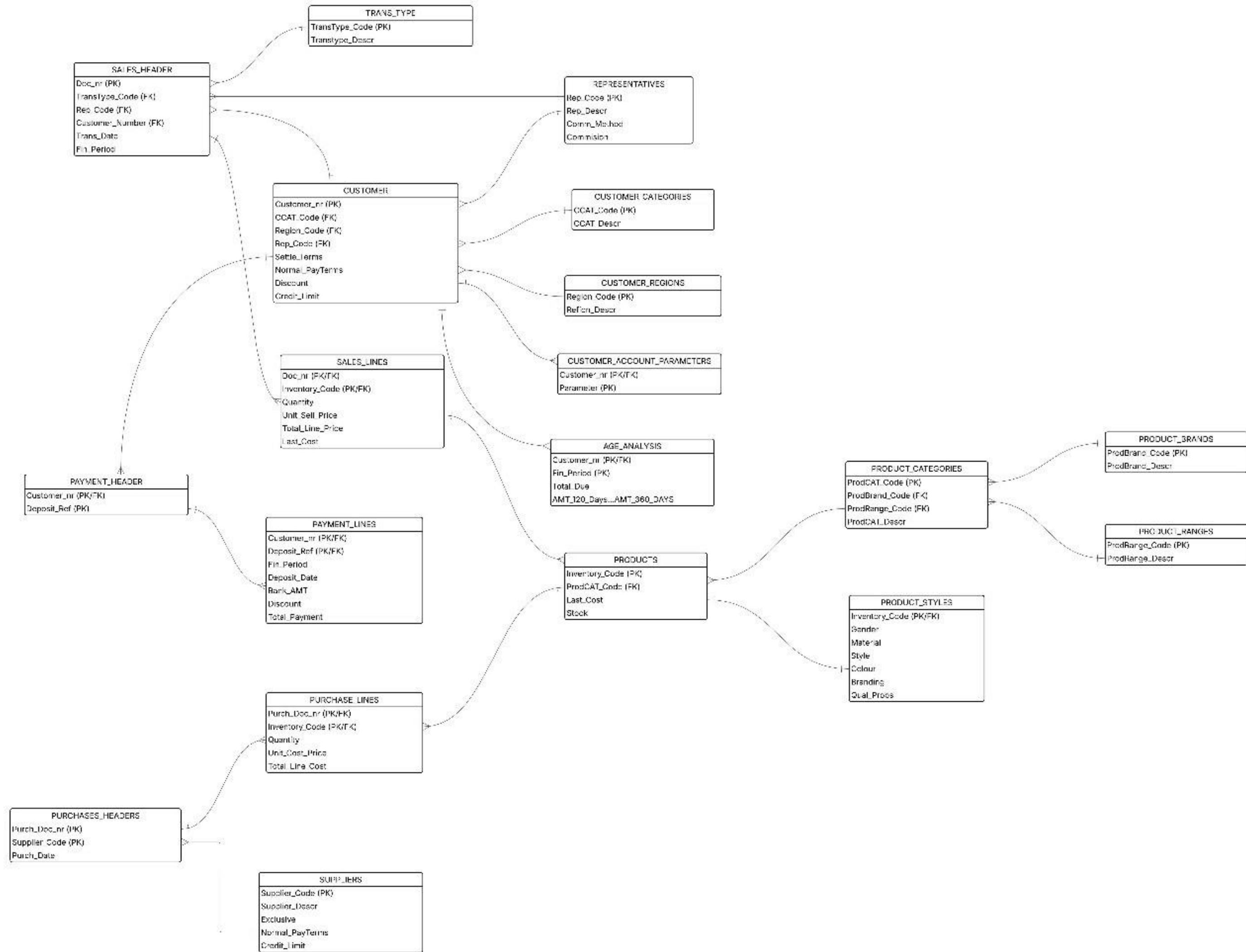
DEBT COLLECTION:

```
{

  "customerID": "599000",

  "depositRefs": ["DB04-153", "DB05-036", "DB05-061"]

}
```



Data Sources

- Sales Transactions (DOC_NUMBER, TRANSTYPE, REP, CUSTOMER, DATE, FIN_PERIOD)
- Customer Master (CUSTOMER_NUMBER, CCAT_CODE, REGION_CODE, CREDIT_LIMIT, TERMS)
- Product Master (PRODUCT_ID, Category, Range, Price)
- Deposits (Customer Deposits Records)



ETL Process

- Extract from ERP
- Transform: clean REP codes, convert dates, validate customer IDs
- Load into MongoDB collections (Sales, Customers, Products, Deposits)



NoSQL Data Warehouse (MongoDB)

- Sales Collection (transactions + line items)
- Customers Collection (profile + credit + deposits)
- Products Collection (reference master)
- Deposits Collection (optional separate or embedded in customers)



BI / Analytics Dashboard

- Daily & Monthly Sales per REP / Region
- Financial Period Revenue Trends
- Credit vs Deposits per Customer
- Transaction Type Analysis



NoSQL Consumption Layer

- MongoDB Aggregation Pipelines
- Ad-hoc queries
- Predictive models for sales & credit risk

9.AI Usage Log

Prompt	AI Prompt (Summary)	AI Output (Summary)	Validation, Improvements & Critical Analysis	Research References / Justification
Requirements Methodology	"Help me draft the Requirements Definition Approach section for a database project..." [Full brief provided]	Provided a generic list of stakeholders and a basic structure involving interviews, data analysis, and research.	Validation: The output was too generic and assumed internal access we don't have. Improvement: We pivoted to a Data Archaeology approach, treating the data files and RFP as primary stakeholders. Role-play was added to compensate for lack of interviews.	The approach was validated by the project constraints and best practices in "Business Analysis for Practitioners: A Practice Guide" by PMI, which emphasizes adapting techniques to project context.
NoSQL Design Patterns	"What are some common data modeling patterns for analytical systems in MongoDB?"	Listed patterns: Embedded Document, Reference, Extended Reference, Bucket Pattern (for time-series), and Computed Pattern.	Validation: Patterns were checked against MongoDB's official documentation. Application: The Embedded Document pattern was selected for sales_header + sales_line and for denormalizing hierarchies. The Extended Reference pattern was chosen for adding frequent product/customer	MongoDB Inc. (2023). <i>Data Modeling Patterns</i> . MongoDB Documentation. https://www.mongodb.com/docs/manual/applications/data-models/
Financial Calendar Logic	"How do you handle a custom financial year like the one starting on the last Saturday of a month?"	Suggested: 1) A calendar lookup table, 2) Implementing the logic algorithmically in code.	Validation: Both are valid. Critical Analysis: During data profiling, we discovered the FIN_PERIOD field already exists in key tables. This finding made option 1 (a lookup table) the superior choice. The ETL process will focus on mapping dates to this existing code, not recalculating the complex logic	This decision was validated by the actual data source, which is the highest authority. It aligns with the Kimball Group's methodology of using a dedicated Date dimension table. Kimball, R., & Ross, M. (2013). <i>The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling</i> (3rd ed.). Wiley.
Data Flow Diagram	Asked for high-level DFD for ETL → BI.	Gave structured stepwise DFD (Data Source → ETL → MongoDB → BI → Queries).	Clear and valid; improvement would be a diagram (visual representation).	ETL Best Practices (Inmon), Data Warehouse Design Principles.
Conceptual NoSQL Design	Asked for a conceptual NoSQL design with Data Flow Diagram (Data Source → ETL → DW → BI → NoSQL).	Provided structured data flow explanation with collections and textual DFD.	Output correct; could be improved with a visual diagram for clarity.	MongoDB Documentation (Schema Design Best Practices), Kimball's Data Warehouse Toolkit.

1).REFERENCES

- Data Warehousing Fundamentals: A Comprehensive Guide for IT Professionals by Paulraj Ponniah.
 - Our own source data analysis, which is the primary reference. |
- | Stakeholder Analysis | "Who are the key stakeholders for a BI system project for sales and financial reporting?" | Listed generic roles: CFO, Sales Director, Data Analysts, IT Admin. | Validation: Roles were correct but not specific to ClearVue. Improvement: We mapped these generic roles to ClearVue's specific data assets. E.g., "Sales Director" needs reports from sales_header and customer_regions; "CFO" needs reports from payment_lines and age_analysis. This grounded the stakeholder needs in the project's reality. | This technique of deriving requirements from available artifacts is supported in:
- IIBA. (2015). A Guide to the Business Analysis Body of Knowledge® (BABOK® Guide) (3rd ed.). Chapter 10. |

Sys design doc - Lecture notes 4 - SYSTEM DESIGN DOCUMENT Overview The System Design Document - Studocu